

INDUSTRIAL SAFETY COMPLIANCE

PPE Object Detection System

Helmet & Safety Vest Monitoring with YOLO11s

Realised by :

- ❖ Hassi Imad-Eddine
- ❖ Bouri Zahia

Supervisor :

- ❖ Dr. Saidi Samira

Automated Real-Time PPE Compliance Monitoring for Factory Environments

Academic Year 2025 / 2026

Deep Learning | Computer Vision | Workplace Safety

1. Executive Summary

This report documents the development of a real-time PPE compliance monitoring system built for industrial factory environments. The system uses the YOLO11s object detection architecture to automatically detect whether workers are wearing their required safety helmets and high-visibility vests. It is intended for integration into existing CCTV infrastructure and provides instant alerts whenever a violation is detected.

The model was trained on a custom-labeled dataset of around 500 to 600 images and achieved an overall mAP50 of 0.749 after 40 epochs of training on a Google Colab T4 GPU. Three classes were defined for detection: Helmet, Safety Vest, and Person. Performance analysis showed that the model is already functional for pilot deployment, though some refinements are still needed before it can be considered a fully reliable safety system.

Main outcomes of the project:

- Real-time detection of helmets and safety vests on factory workers
- mAP50 of 0.749 with precision reaching 1.00 at high confidence thresholds
- A custom dashboard built with customtkinter for live camera monitoring
- Identification of specific weaknesses (helmet miss rate, background false positives) with a clear improvement plan
- Ready for pilot testing in a real industrial environment

2. Project Overview

2.1 Problem Statement

In manufacturing and heavy industry, PPE compliance is one of the most basic requirements for worker safety. The problem is that monitoring compliance manually across a large factory floor with multiple camera feeds is not practical. Supervisors cannot watch every worker at every moment, and violations often go unnoticed until after an incident has already occurred.

The goal of this project was to automate that monitoring process using a trained object detection model, so that any worker found without a helmet or safety vest triggers an immediate alert, without requiring any human to watch the cameras continuously.

2.2 Proposed Solution

The solution developed here is a computer vision pipeline that processes live video from standard IP cameras and applies a trained YOLO11s model to each frame. For every worker detected in the frame, the system checks whether they are wearing the required PPE and flags any violation immediately. The key components of the system are:

- A trained YOLO11s model capable of detecting Helmet, Safety Vest, and Person in real time
- A video processing pipeline using OpenCV to read camera streams frame by frame
- A desktop dashboard built with customtkinter showing live detections with color-coded alerts
- Support for both live webcam feeds and recorded video files for retrospective auditing

2.3 Target Deployment Environments

- Manufacturing plants and assembly line floors
- Construction and civil engineering sites
- Logistics warehouses and loading zones
- Oil and gas facilities and chemical plants
- Any industrial site where helmet and vest compliance is mandatory

3. Project Methodology

This section describes the full workflow that was followed to build the system, from collecting images all the way to running live inference on a local machine.

Step	Description
01	Gather the Dataset Started by collecting 500 to 600 images showing workers in various factory and construction settings, captured under different lighting conditions, distances, and camera angles. Some images were taken manually and others were sourced from Roboflow Universe and Kaggle.
02	Organize Files Moved all images into a single organized folder on the local machine (e.g., YOLO/ppe_images/) so that the labeling tool and training scripts could access them cleanly without confusion.
03	Set Up Labeling Environment Installed Anaconda to manage Python environments, created a dedicated environment for the project, and then installed Label Studio inside it. Label Studio is an open-source annotation tool that makes drawing bounding boxes straightforward.
04	Label the Images Launched Label Studio, created an Object Detection project, imported the images, and manually drew bounding boxes around every helmet, vest, and person visible in each image. Each box was assigned to the correct class.
05	Export Labeled Data Once labeling was complete, exported the annotations in YOLO format. This generated the required folder structure with images/ and labels/ subfolders and a classes.txt file. Everything was then packaged into a data.zip file for upload.
06	Prepare Google Colab Opened a Google Colab notebook and switched the runtime to use a T4 GPU (available for free). Verified that the GPU was available and working using nvidia-smi before starting any training.
07	Upload and Split the Data Uploaded data.zip to Colab and unzipped it. A Python script was then used to automatically split the dataset into 90% for training and 10% for validation, making sure both sets had a reasonable spread of classes.

Step	Description
08	Configure the Model Installed the Ultralytics library using pip and generated a data.yaml file that told the model where to find the training and validation folders and what the class names were.
09	Select Training Parameters Chose YOLO11s as the architecture because it is lightweight and fast enough for real-time edge deployment. Set the number of epochs to 40 and the image resolution to the standard 640x640 pixels.
10	Train the Model Ran the training command and watched the metrics update epoch by epoch. Kept an eye on mAP, precision, and recall to check that the model was actually learning and not overfitting or stalling.
11	Test and Evaluate After training finished, ran inference on the validation folder to see how well the model detected objects compared to the ground truth labels. Also reviewed the confusion matrix and performance curves to understand where the model was struggling.
12	Deploy Locally Downloaded the best.pt weights file from Colab and set up a local environment on a personal PC using Anaconda and Ultralytics, matching the same Python and library versions used during training.
13	Run Live Inference Wrote a Python script using OpenCV and the Ultralytics library to run the model on a live webcam feed and on recorded video files. Built a desktop GUI with customtkinter so the results can be viewed clearly by factory supervisors.

4. Dataset

4.1 Dataset Description

The dataset was built specifically for this project by combining manually captured images with images found on publicly available platforms like Roboflow Universe and Kaggle. The images were chosen to cover as many different factory scenarios as possible, including different lighting conditions (bright outdoor sites, dim indoor floors), various camera angles (overhead, eye-level, side-angle), and different numbers of workers in the same frame.

A total of around 500 to 600 images were collected and annotated. Three classes were defined: Helmet, Person, and Vest. The full dataset was split into 90% for training and 10% for validation. Labels were created manually using Label Studio, with bounding boxes drawn around every visible instance of each class.

4.2 Class Distribution

Looking at the labels.jpg figure below, the dataset has a noticeable class imbalance. The Person class has 2,196 instances, Helmet has 821, and Vest only has 465. This means the Person class is

almost five times more common than the Vest class. This imbalance has a real impact on training because the model gets much more exposure to Person examples than to Vest examples, which can cause it to be better at detecting people than vests. This is addressed in the future work section.

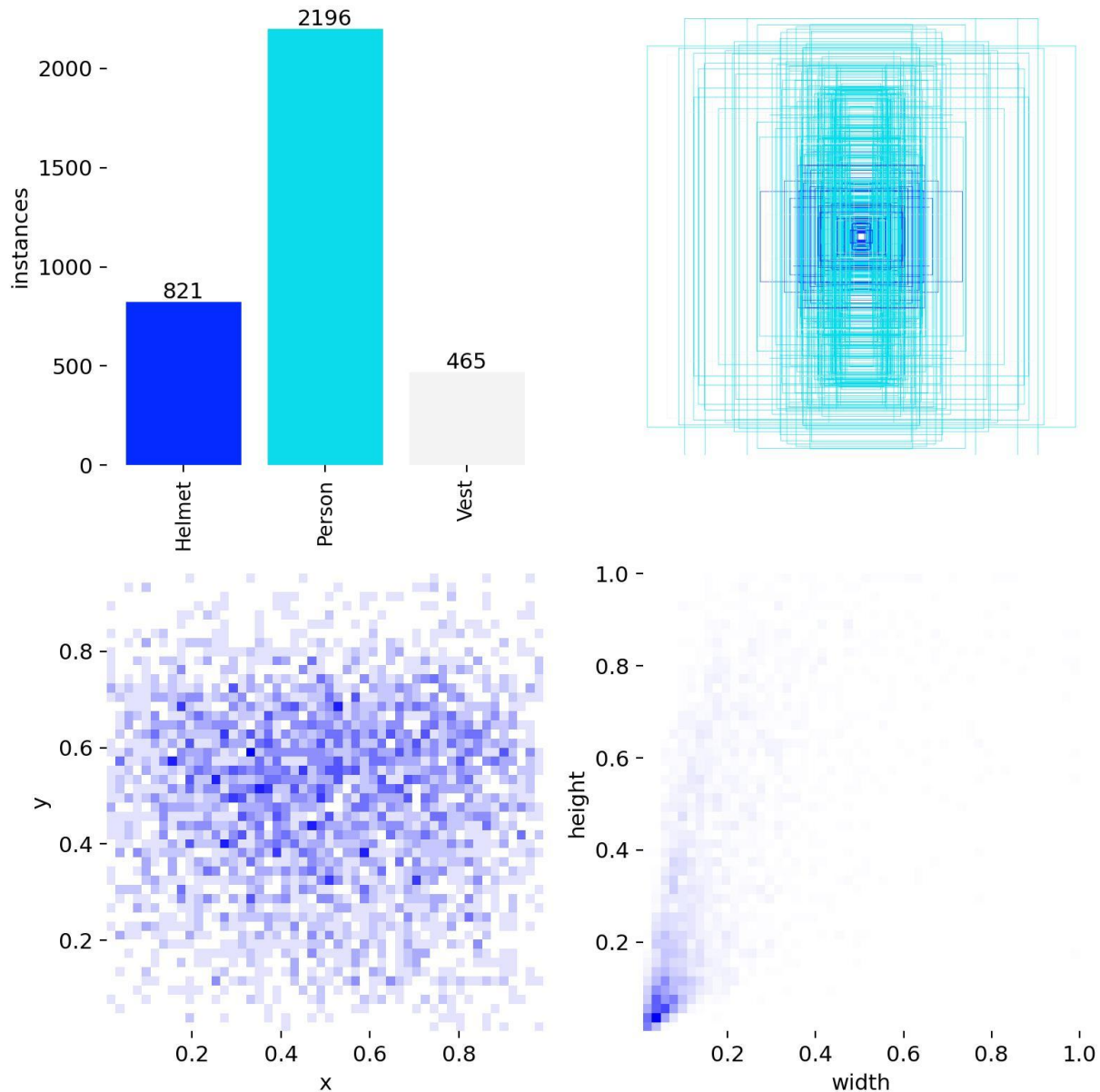


Figure 1: Class instance counts and bounding box spatial distribution from the training dataset. Person (2,196 instances) clearly dominates over Helmet (821) and Vest (465), which creates a class imbalance that affects model sensitivity.

4.3 Training Batch Examples

The figures below show samples from the first training batches (batch 0, 1, and 2). These batches illustrate the mosaic augmentation technique used by YOLO11s, where four images are combined into a single training tile. This helps the model learn to detect small objects and handle crowded scenes.

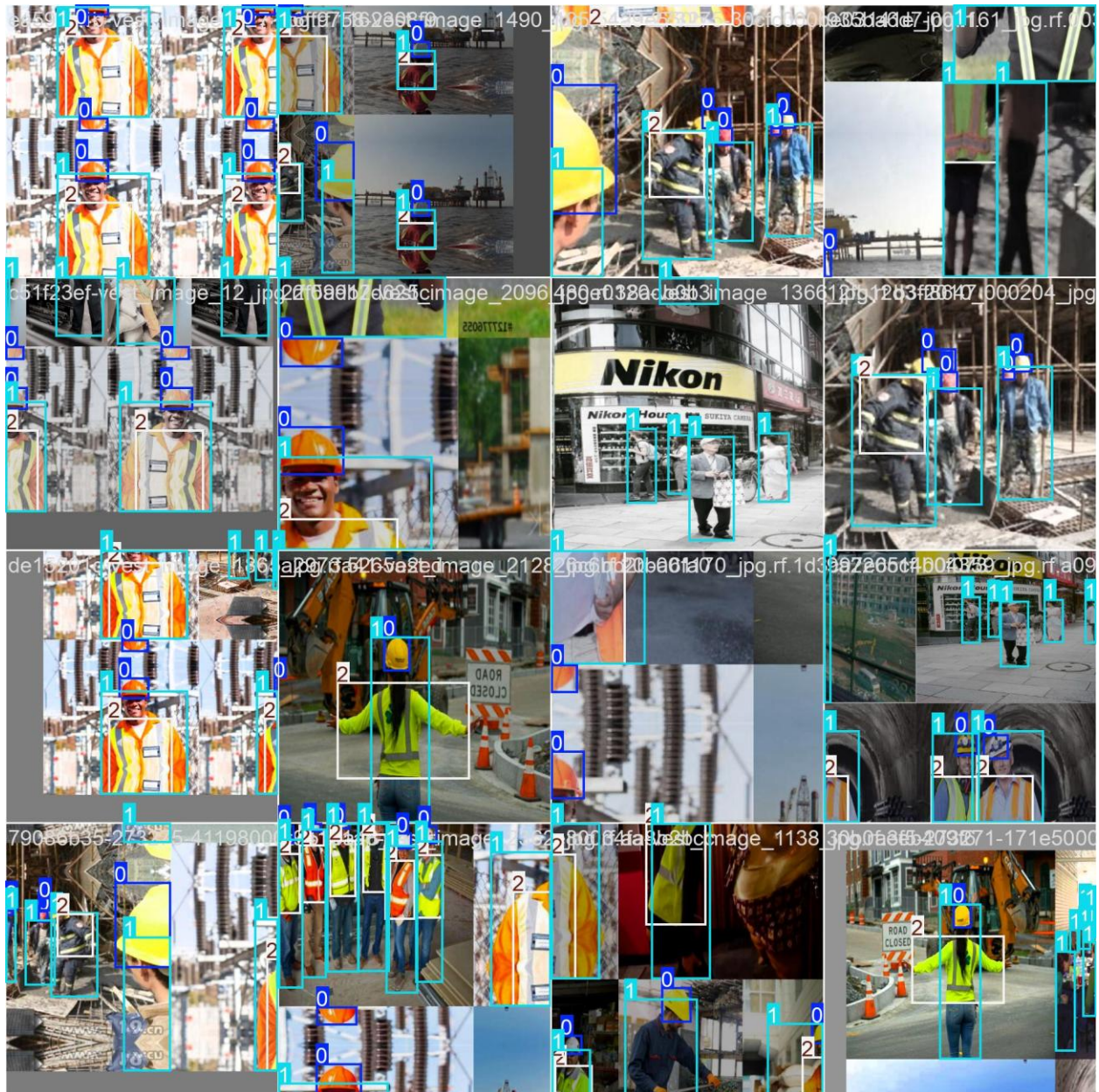


Figure 2: Training batch 0 showing the mosaic augmentation applied to the first group of training images. Each tile combines four annotated images.



Figure 3: Training batch 1 showing a different combination of images with varied backgrounds, distances, and lighting.



Figure 4: Training batch 2 showcasing further diversity in the training data including construction sites and industrial environments.

The figures below show batches from later in training (batch 870, 871, 872), which represent the kind of data the model was still processing near the final epochs. The variety here is important because it shows the model was not just memorizing a small set of images.

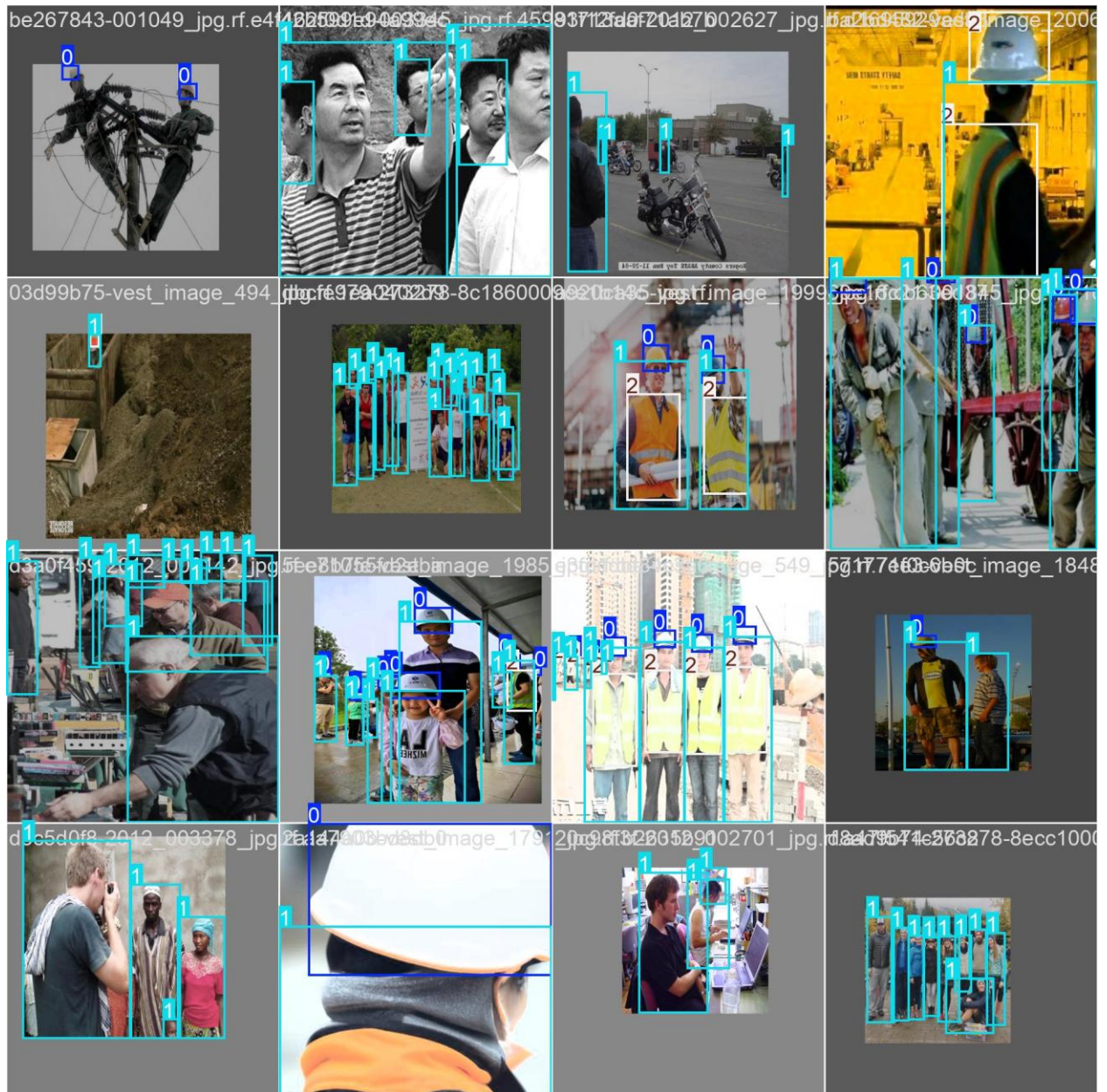


Figure 5: Training batch 870 from late in the training run, showing continued diversity and augmentation variation.

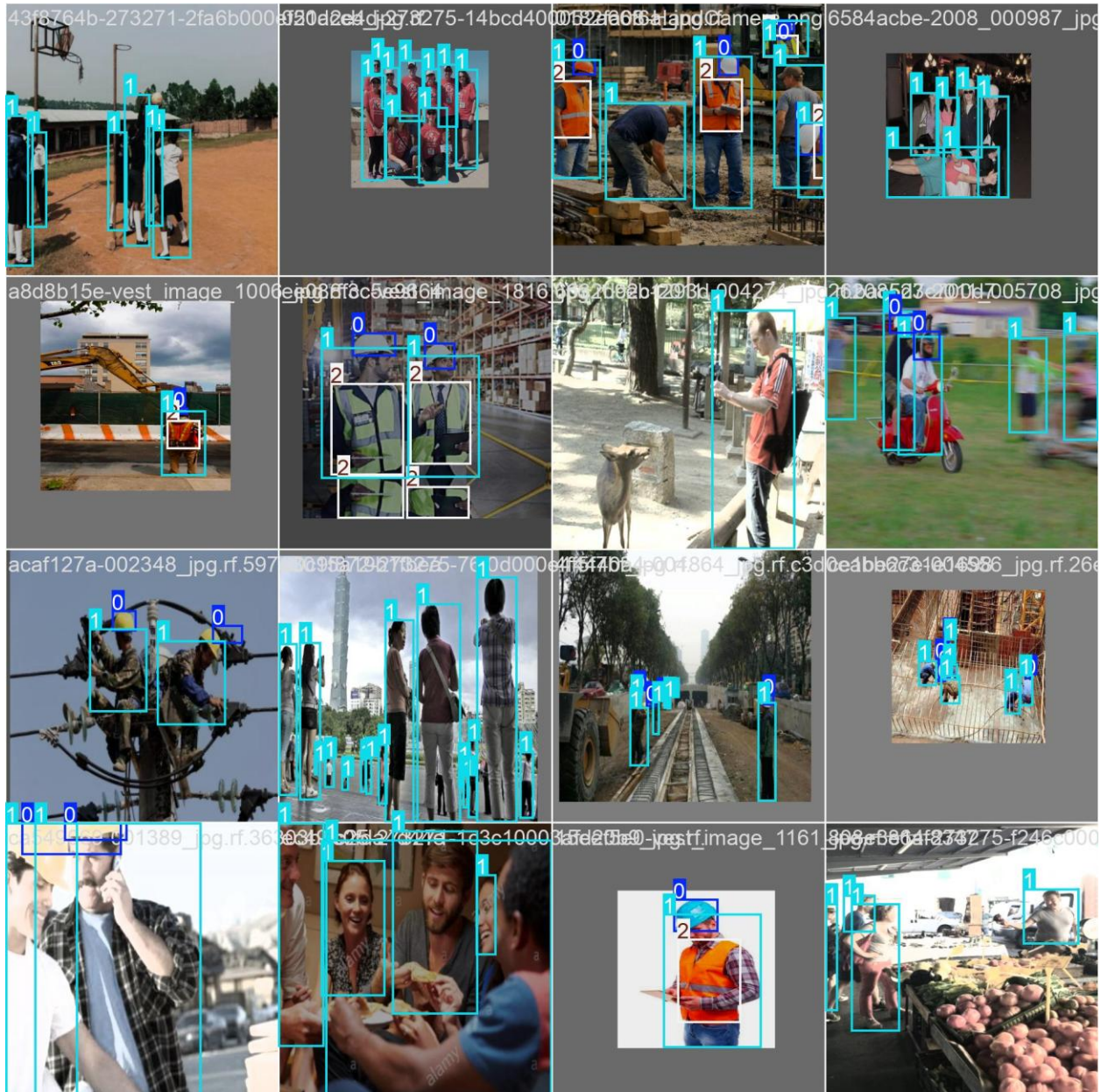


Figure 6: Training batch 871 with additional scene variety including different worker poses and equipment configurations.

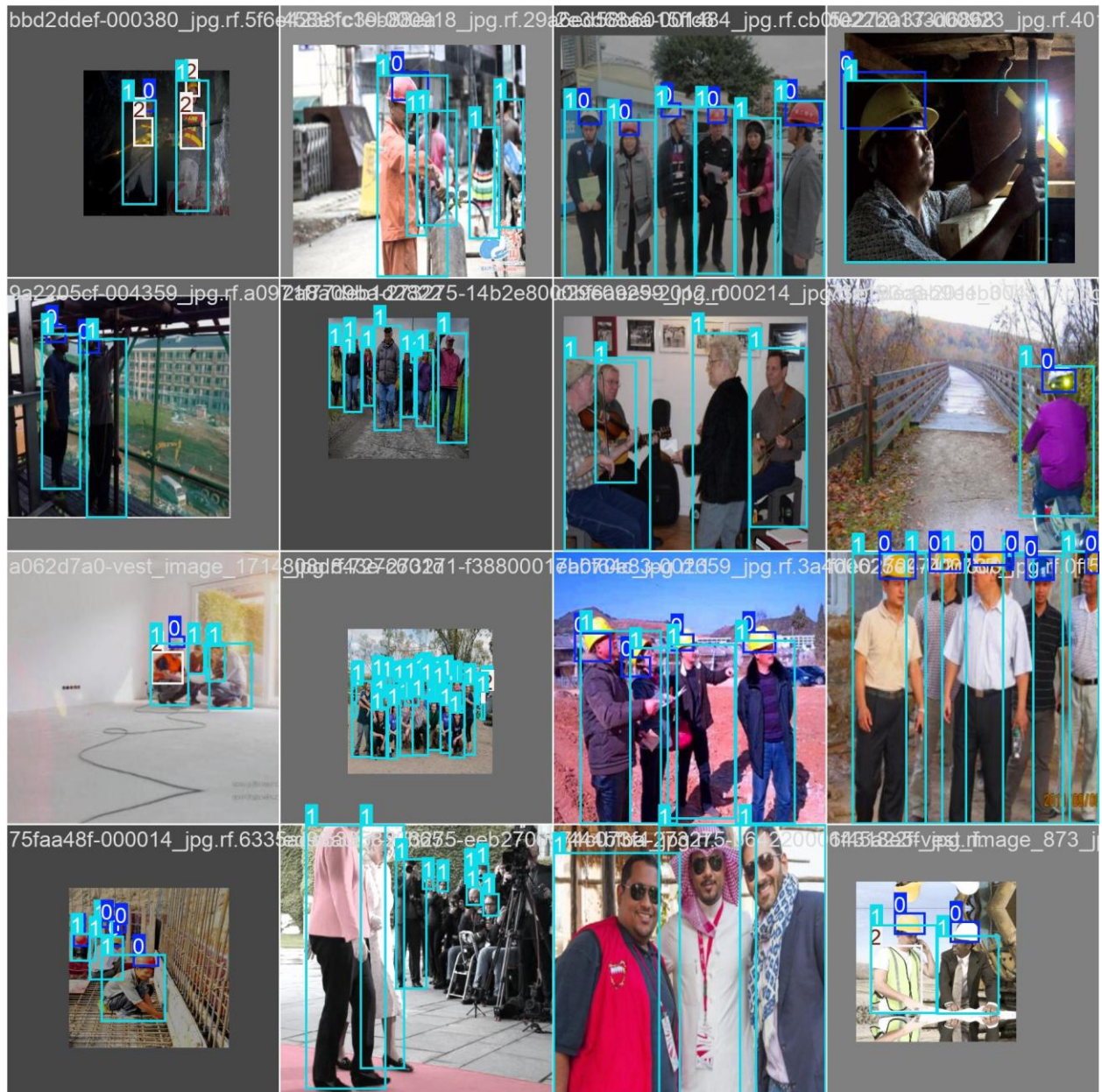


Figure 7: Training batch 872, one of the final batches processed during training.

5. Model Architecture and Training Setup

5.1 Why YOLO11s

YOLO11s (You Only Look Once, version 11, small variant) was selected for this project because it offers a good balance between speed and accuracy. The small variant is lightweight enough to run on a standard GPU or even a decent CPU without significant lag, which is important for real-time deployment on factory floor hardware. It also uses anchor-free detection and a decoupled head architecture, which generally gives cleaner predictions on multi-scale targets like helmets (small) and vests (medium to large).

5.2 Training Configuration

Metric	Value	Notes
Architecture	YOLO11s	Small variant, anchor-free detection
Hardware	Google Colab T4 GPU	Free-tier runtime used for all training
Input Resolution	640 x 640 pixels	Standard YOLO input size
Data Split	90% Train / 10% Validation	Around 500 to 600 total images
Number of Epochs	40	Training completed without early stopping
Number of Classes	3	Helmet, Person, Vest
Pretrained Weights	YOLO11s.pt	COCO pretrained initialization
Optimizer	SGD with cosine LR decay	Final LR approximately 4.97e-5
Confidence Threshold	0.25	Used during validation inference
IOU Threshold	0.45	Used for non-maximum suppression

6. Training Results and Performance Analysis

6.1 Loss and Metric Curves

The figure below shows ten subplots covering the main training and validation metrics across all 40 epochs. All three loss components (box loss, classification loss, and DFL loss) decrease consistently on both the training and validation sets, which is a healthy sign that the model is actually learning without severe overfitting. The mAP50 on the validation set climbs steadily from around 0.1 in the first few epochs to a final value of 0.749 by epoch 40, and it was still trending slightly upward at the end, suggesting that additional epochs could improve performance further.

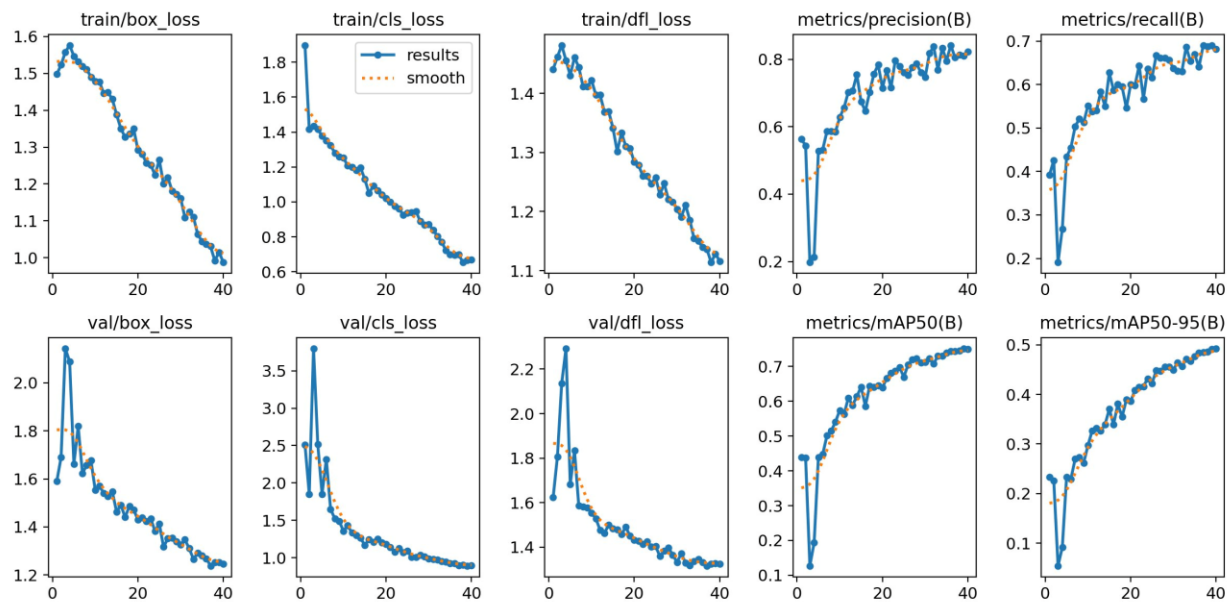


Figure 8: Full training results showing loss curves (box, cls, DFL) and performance metrics (precision, recall, mAP50, mAP50-95) for both training and validation sets across 40 epochs.

6.2 Final Performance Metrics

Metric	Value	Notes
mAP50 (all classes)	0.749	Primary benchmark metric
mAP50-95 (all classes)	0.492	Stricter COCO-style localization metric
mAP50 Helmet	0.685	Lowest class score due to small object size
mAP50 Person	0.787	Highest score, most training instances
mAP50 Vest	0.776	Good score despite fewer training examples
Best Precision (all)	1.00 at conf=0.931	All predictions correct at very high threshold
Best Recall (all)	0.87 at conf=0.000	87% of all objects found at zero threshold
Best F1 Score (all)	0.74 at conf=0.359	Optimal balance point for practical use

6.3 Precision-Recall Curve

The PR curve shows the trade-off between precision and recall for each class. Person achieves the best area under the curve (0.787), followed closely by Vest (0.776). Helmet lags behind at 0.685,

which is consistent with the expectation that small objects are harder to detect reliably at 640x640 resolution. The overall mAP50 of 0.749 confirms that the model is working reasonably well across all three classes.

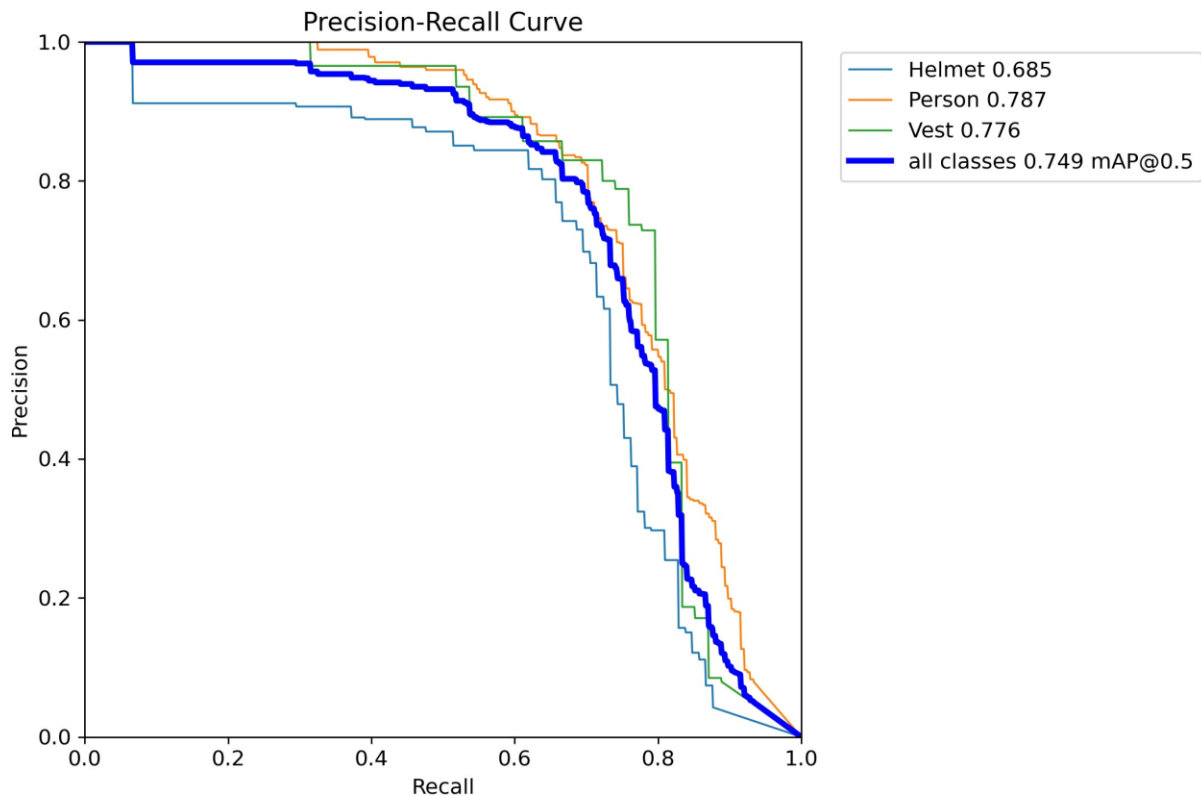


Figure 9: Precision-Recall curve per class. Helmet (mAP 0.685) underperforms relative to Person (0.787) and Vest (0.776), primarily due to its smaller spatial footprint in the image.

6.4 Precision-Confidence Curve

This curve shows how precision changes as the confidence threshold is raised. Precision reaches 1.00 for all classes at a threshold of 0.931. This means that at very high confidence, the model almost never produces a false positive. The downside is that recall drops significantly at high thresholds, so the model will miss more objects. For a safety application, the confidence threshold needs to be tuned carefully to balance precision and recall.

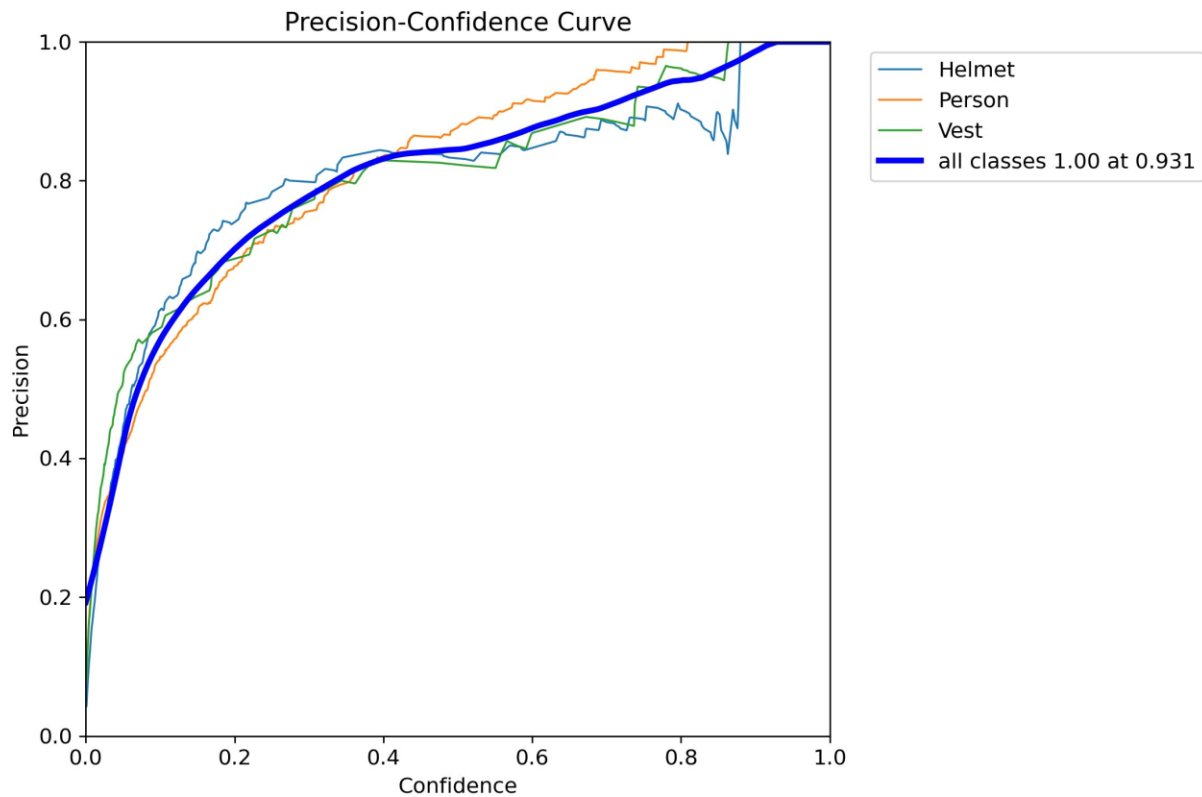


Figure 10: Precision-Confidence curve showing that all classes reach precision = 1.00 at a threshold of 0.931. Precision for Person rises fastest, while Helmet trails slightly.

6.5 Recall-Confidence Curve

The recall curve starts at 0.87 for all classes at zero confidence and drops as the threshold increases. This shows that if the threshold is set too high, the system will start missing real violations. At a threshold of 0.6, recall has already dropped below 0.6 for some classes, meaning more than 40% of actual violations would be missed. This is one of the key reasons why threshold tuning is critical before deploying the system in a real factory.

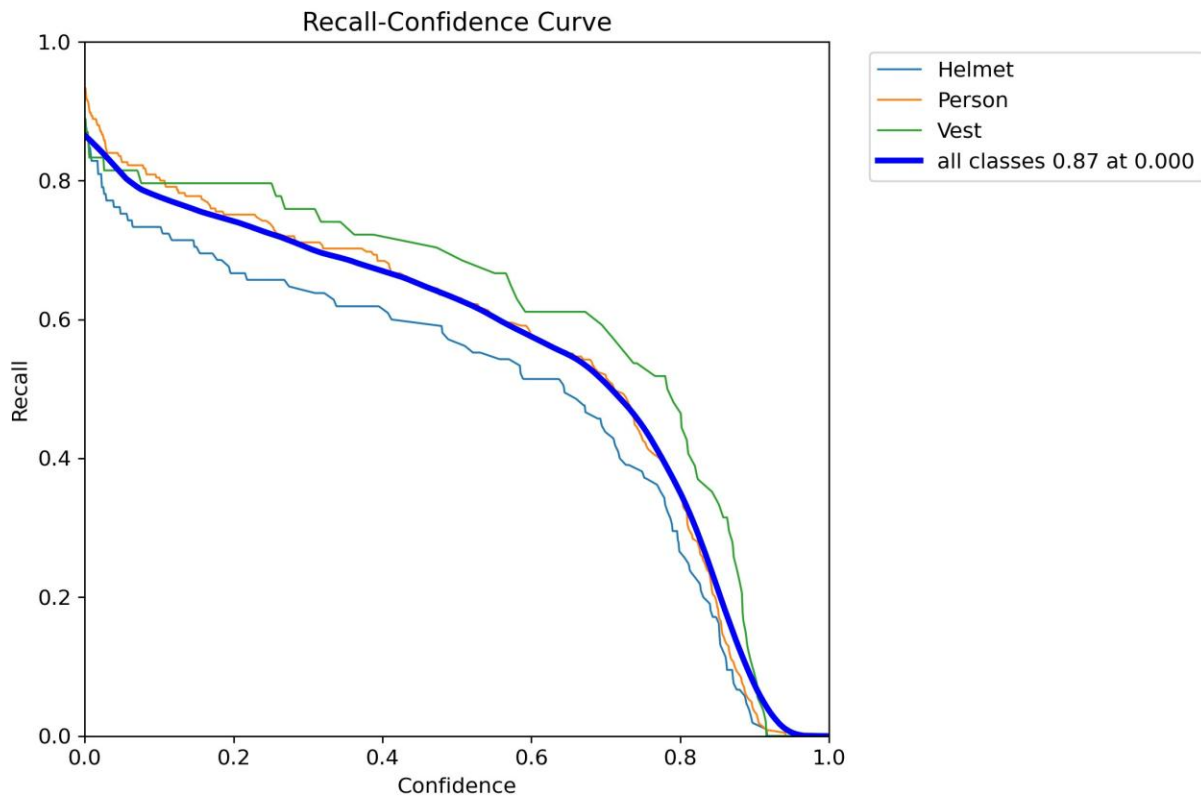


Figure 11: Recall-Confidence curve. The overall recall starts at 0.87 and degrades as confidence increases, with Helmet recall dropping the fastest.

6.6 F1 Score Curve

The F1 curve combines precision and recall into a single score. The best overall F1 score is 0.74, reached at a confidence threshold of around 0.36. This is a useful reference point for setting the operating threshold in the deployed system, since it represents the best balance between catching real violations and not flooding the supervisor with false alarms.

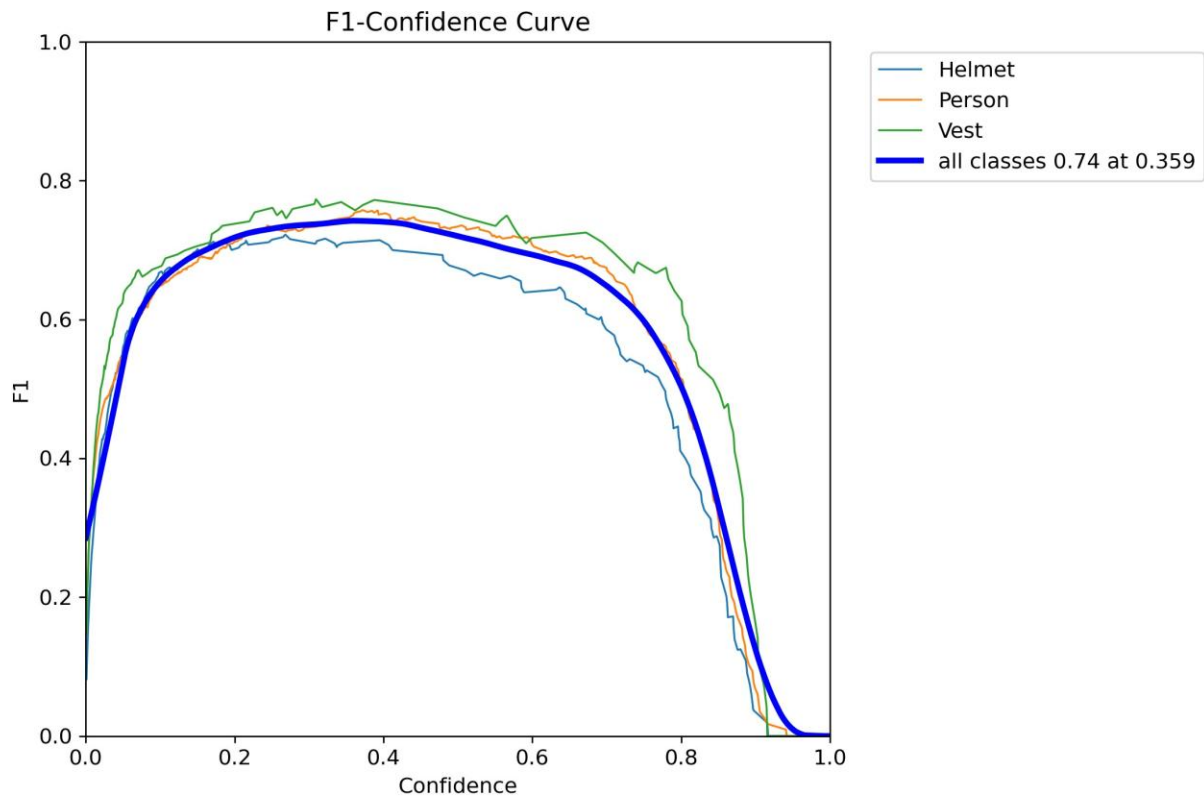


Figure 12: F1-Confidence curve. Peak F1 of 0.74 is achieved at a confidence threshold of 0.359. This is the recommended starting point for the deployment configuration.

6.7 Confusion Matrices

Two confusion matrices were analyzed. The raw count matrix gives a concrete picture of detection volumes, while the normalized version is better for understanding per-class accuracy rates.

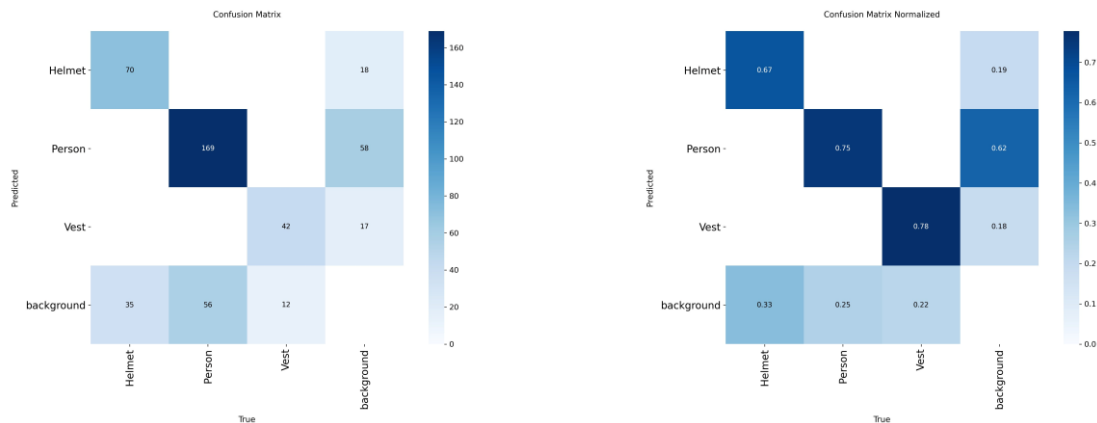


Figure 13: Left: Raw count confusion matrix. Right: Normalized confusion matrix showing per-class detection rates. Key observations are detailed below.

Reading the normalized matrix directly:

- Helmet detection rate: 0.67. One third of helmets (0.33) are missed and classified as background. For a safety application this is a significant gap that needs to be addressed.
- Person detection rate: 0.75. About 25% of actual persons are missed (classified as background), and 62% of background regions are falsely predicted as Person. This high background-to-Person false positive rate is the main driver of alarm fatigue risk.
- Vest detection rate: 0.78. This is the best result among the three classes, which is somewhat surprising given that Vest has the fewest training instances. The larger physical size of vests in the frame likely helps here.

The 0.33 helmet miss rate is the most serious finding from a safety perspective. Missing a third of helmet violations would significantly undermine the purpose of the system.

7. Detection Results on Validation Set

7.1 Validation Batch 0: Ground Truth vs Predictions

The two images below show the same validation batch with ground truth labels on the left and model predictions on the right. Comparing them side by side is the clearest way to see where the model agrees with the labels and where it makes mistakes.



Figure 14: Validation batch 0 with ground truth bounding box annotations. Labels show the correct positions of Helmet, Person, and Vest instances as manually annotated.

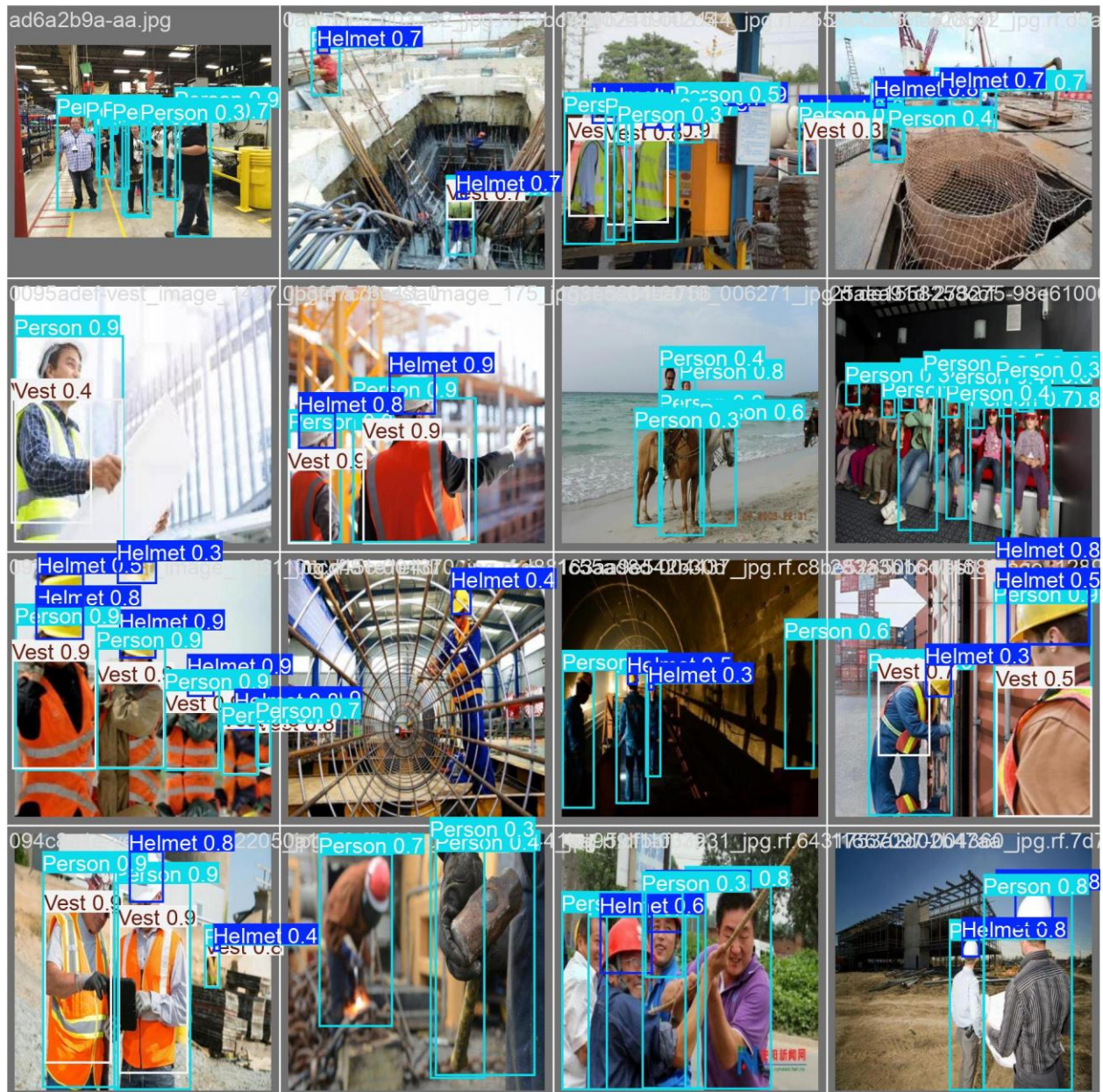


Figure 15: Validation batch 0 with model predictions and confidence scores. Most detections align well with ground truth, though some helmets at a distance and partially occluded vests are missed.

7.2 Validation Batch 1: Ground Truth vs Predictions



Figure 16: Validation batch 1 with ground truth labels showing the diversity of scenes including indoor factory floors, construction sites, and crowded multi-worker images.

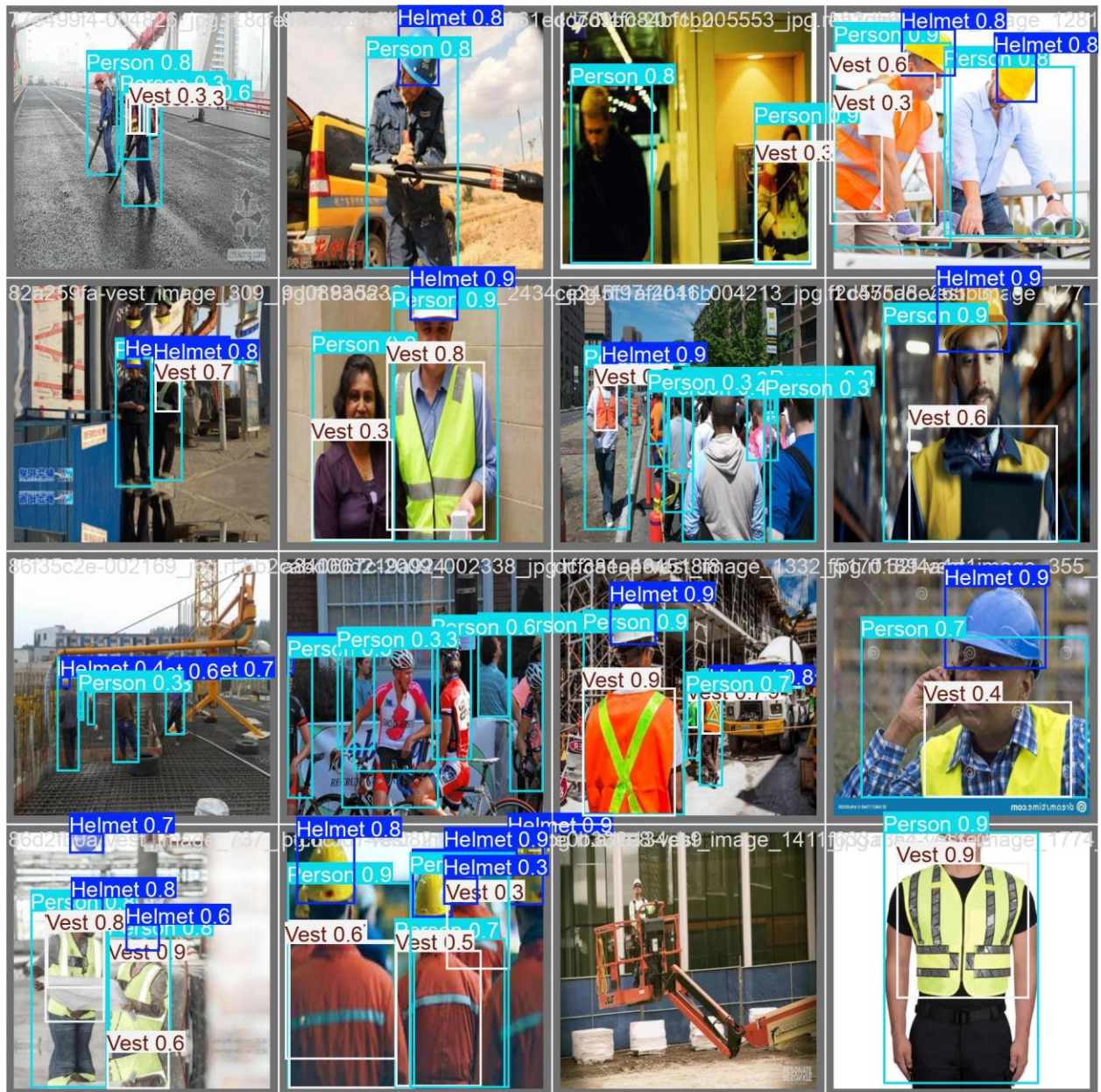


Figure 17: Validation batch 1 with model predictions. The model handles most scenes well, but shows some over-detection of Person class in cluttered backgrounds.

8. System Deployment and Real-Time Interface

8.1 Local Deployment Setup

After downloading the best.pt weights from Colab, the model was deployed locally on a standard PC. The local environment was set up using Anaconda with the same Python version and Ultralytics library version used during training to avoid any compatibility issues. OpenCV was used to handle video capture and frame preprocessing before passing frames to the model.

8.2 Dashboard Interface

A desktop monitoring dashboard was developed using the customtkinter library. The interface was designed to be practical and readable in an industrial environment, using a dark theme with high contrast colors so that alerts are clearly visible even under factory floor lighting.

The dashboard supports two operating modes:

- Live webcam mode: processes frames from a connected camera in real time, suitable for active shift monitoring
- Video file mode: loads and processes a recorded video file, useful for reviewing past footage and auditing compliance

For each detected object, the system draws a labeled bounding box with the class name and confidence score. When a violation is detected (worker without helmet or vest), the bounding box is highlighted in red and an alert is logged with a timestamp.

8.3 Hardware and Software Used

Metric	Value	Notes
Training GPU	Google Colab T4	Used for all model training
Inference GPU	Standard laptop/desktop GPU	RTX 3050 or equivalent
Camera Input	USB webcam or IP camera via RTSP	Supports most standard cameras
Framework	Ultralytics YOLO11s with PyTorch	Python 3.10+
GUI Framework	customtkinter	Dark themed desktop dashboard
Video Processing	OpenCV 4.x	Frame capture and preprocessing
Logging	Local file or database	Timestamps and violation snapshots

9. Industrial Impact and Benefits

9.1 Safety Impact

The most direct benefit of this system is that it removes the human factor from PPE monitoring. A supervisor watching multiple camera feeds will inevitably miss violations, especially during busy periods or long shifts. An automated system watching every frame at every moment provides a level of consistency that manual monitoring simply cannot match.

- 24/7 monitoring without fatigue or attention loss

- Immediate alerts for violations, reducing response time from minutes to seconds
- Consistent enforcement across all camera zones and all shifts
- A permanent record of violations that can be used for safety audits and regulatory compliance
- Documented evidence for inspections under standards like OSHA 1910.132 and ISO 45001

9.2 Operational Benefits

- Can be integrated into existing CCTV infrastructure without major hardware changes
- Scales easily to any number of cameras just by adding more processing nodes
- Frees up safety supervisors to focus on active risk management rather than passive monitoring
- Generates useful data over time about which zones have the most violations and at what times

9.3 Return on Investment

Metric	Value	Notes
System Implementation	Low to medium cost	Uses existing camera infrastructure
Workplace Injury Cost (average)	Over \$40,000 per incident	Direct and indirect costs combined
OSHA Citation (per violation)	Up to \$15,625	Repeat violations can be much higher
Estimated Payback Period	6 to 18 months	Depends on facility size and violation rates
Scalability	High	One trained model covers all cameras

10. Conclusion and Future Work

10.1 Summary

This project produced a working automated PPE compliance monitoring system using YOLO11s. The model was trained from scratch on a custom dataset, evaluated thoroughly using standard metrics, and deployed in a functional desktop application. With an overall mAP50 of 0.749 and a precision of 1.00 at high confidence thresholds, the system is technically viable and ready for pilot testing in a real industrial environment.

That said, the analysis also identified two important issues that need to be resolved before the system can be used as a primary safety tool. The helmet miss rate of 0.33 and the high background-to-Person false positive rate are both problems that would reduce trust in the system if left unaddressed. These are known and solvable issues, and the next phase of the project will focus specifically on them.

10.2 Planned Improvements

The three priority improvements identified from the performance analysis are:

Hard Negative Mining:

To fix the high rate of background regions being misclassified as Person, a set of negative images (empty factory floors, machinery without workers) will be added to the training data. These hard negatives teach the model what a non-person scene looks like.

Improving Helmet Detection:

The 0.33 miss rate for helmets is the most critical issue. The plan is to add more helmet-focused training images, including zoomed-in crops of helmets from various angles and distances. Higher-resolution tiling during inference may also help with very small helmet instances.

Dataset Rebalancing:

With only 465 Vest instances compared to 2,196 Person instances, the dataset needs more vest examples to bring the class distribution closer to balance. Targeted data collection focusing on workers wearing vests will be the next data gathering step.

Additional future enhancements that could be added in later versions:

- Expanding PPE detection to cover gloves, safety goggles, and steel-toed boots
- Zone-based compliance rules where different areas of the factory require different PPE
- Person tracking across multiple cameras to follow individual workers
- A mobile alert application so supervisors receive notifications on their phone
- Low-light and night shift model variants for factories that operate around the clock

10.3 Final Notes

This project was developed as part of academic coursework in the 2025/2026 academic year. The YOLO11s framework provided by Ultralytics was the main technical foundation. The work was guided by real industrial safety requirements and aims to contribute to the broader goal of reducing workplace accidents through practical AI applications.

Appendix: Technical Reference

A. Training Command

```
yolo train model=yolo11s.pt data=data.yaml epochs=40 imgsz=640 batch=16
```

B. Inference Command

```
yolo predict model=best.pt source=0 conf=0.36 iou=0.45 show=True
```

C. data.yaml Configuration

```
path: ./ppe_dataset
train: images/train
val: images/val
nc: 3
names: ['Helmet', 'Person', 'Vest']
```

D. References

- Ultralytics YOLO Documentation: <https://docs.ultralytics.com>
- Redmon, J. et al. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016.
- OSHA Standard 29 CFR 1910.132: Personal Protective Equipment requirements.
- ISO 45001:2018: Occupational Health and Safety Management Systems.
- ILO World Safety and Health at Work Report, 2023.
- Roboflow Universe: <https://universe.roboflow.com> (training data source)